

SIMATS ENGINEERING
ASSESSMENT TOOL 2
COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools. (BL4, BL5)

Assessment Tool Weightage: 30%

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I Harish Kumar G (Reg No: 192521340) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Name: _____ **Register Number:** _____

Question 1

In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values, including validation, classification, and a summary of network performance.

1.1 Problem Understanding

- Given (Inputs): Total data received = 8000 bits, Time = 4 seconds, Packets sent = 200, Packets received = 180.
- Required (Outputs): Throughput (bps), Packet Loss (%), and a classification of the network as Efficient or Inefficient.
- Constraints: Time and Packets Sent must never be zero, otherwise a division-by-zero error occurs.
- Formulae used: Throughput = Total Data Received / Time; Packet Loss % = ((Packets Sent - Packets Received) / Packets Sent) × 100.

1.2 Pseudocode

```
ALGORITHM CalculateNetworkPerformance
BEGIN
    // Step 1: Declare and take inputs
    READ total_data_received_bits      // e.g., 8000 bits
    READ time_seconds                  // e.g., 4 seconds
    READ packets_sent                  // e.g., 200
    READ packets_received              // e.g., 180

    // Step 2: Input validation to prevent division-by-zero errors
    IF time_seconds = 0 THEN
        DISPLAY "Error: Time duration cannot be zero."
        STOP
    END IF

    IF packets_sent = 0 THEN
        DISPLAY "Error: Total packets sent cannot be zero."
        STOP
    END IF

    // Step 3: Calculate throughput (bits per second)
    throughput = total_data_received_bits / time_seconds

    // Step 4: Calculate packet loss percentage
    packets_lost = packets_sent - packets_received
    IF packets_lost < 0 THEN
        packets_lost = 0                // safeguard against invalid data
    END IF
    packet_loss_percent = (packets_lost / packets_sent) * 100

    // Step 5: Classify network performance
```

```

IF throughput > 1500 AND packet_loss_percent < 10 THEN
    network_status = "Efficient"
ELSE
    network_status = "Inefficient"
END IF

// Step 6: Display complete performance summary
DISPLAY "===== NETWORK PERFORMANCE SUMMARY ====="
DISPLAY "Total Data Received   : ", total_data_received_bits, " bits"
DISPLAY "Time Duration          : ", time_seconds, " seconds"
DISPLAY "Throughput              : ", throughput, " bps"
DISPLAY "Packets Sent              : ", packets_sent
DISPLAY "Packets Received          : ", packets_received
DISPLAY "Packets Lost              : ", packets_lost
DISPLAY "Packet Loss               : ", packet_loss_percent, " %"
DISPLAY "Network Status           : ", network_status
DISPLAY "===== "
END

```

1.3 Dry Run (Trace Table) Using the Given Values

Step	Variable	Calculation	Result
1	time_seconds	Given = 4 ($\neq 0$, passes validation)	Valid
2	packets_sent	Given = 200 ($\neq 0$, passes validation)	Valid
3	throughput	$8000 / 4$	2000 bps
4	packets_lost	$200 - 180$	20
5	packet_loss_percent	$(20 / 200) \times 100$	10 %
6	network_status	throughput(2000)>1500 TRUE, but loss(10)<10 is FALSE	Inefficient

Final Output: Throughput = 2000 bps, Packet Loss = 10%, Network Status = Inefficient (because although the throughput condition is satisfied, the packet-loss condition of "strictly less than 10%" is not, since the loss is exactly equal to 10%).

1.4 Explanation

The algorithm begins by reading the four raw inputs collected from the Packet Tracer simulation. Before performing any arithmetic, it validates that the time duration and the number of packets sent are non-zero, since both values act as denominators in the throughput and packet-loss formulae; skipping this check could cause a runtime division-by-zero error. Throughput is computed as the ratio of total data received to the time taken, expressed directly in bits per second, which is the standard unit for measuring the effective data-transfer rate of a link. Packet loss is computed as the percentage of packets that failed to reach the destination out of all packets that were transmitted. A compound conditional statement (AND) is then used to classify the link: only when the throughput exceeds 1500 bps and the packet loss is strictly below 10% is the network labelled Efficient; any other combination is labelled Inefficient. Finally, all intermediate and derived values are displayed together as a single,

human-readable summary block, which makes the algorithm suitable for direct use in a monitoring dashboard or a simulation report.

Question 2

An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each link, stores the values, detects utilization above 80%, and recommends switching traffic to a backup link.

2.1 Problem Understanding

- Given (Inputs): A set of network links, each with a continuously changing bandwidth-utilization percentage.
- Required (Outputs): A stored history of utilization values, an alert whenever a link crosses the 80% threshold, and a recommendation to switch that link's traffic to a backup link.
- Constraints: Monitoring must run continuously (periodic polling), not just once, so a loop with a wait/delay is required.
- Data structure: An array/list indexed by link name is used to store historical utilization readings for trend analysis.

2.2 Pseudocode

```
ALGORITHM MonitorBandwidthUtilization
BEGIN
    // Step 1: Initialization
    DECLARE links[] = {L1, L2, L3, ..., Ln}      // all monitored links
    DECLARE bandwidth_usage[links] = empty list  // history of readings per
link
    DECLARE backup_link[links]                    // pre-assigned backup for
each link
    CONSTANT THRESHOLD = 80                       // percentage
    CONSTANT POLL_INTERVAL = 5 minutes
    monitoring_active = TRUE

    // Step 2: Continuous monitoring loop
    WHILE monitoring_active = TRUE DO
        FOR EACH link IN links DO

            // Step 3: Collect current bandwidth utilization
            usage = GET_CURRENT_UTILIZATION(link)    // returns 0-100 (%)

            // Step 4: Store the value for history / reporting
            APPEND usage TO bandwidth_usage[link]

            // Step 5: Check for overload condition
            IF usage > THRESHOLD THEN
                (overloaded)"
                DISPLAY "ALERT: ", link, " utilization = ", usage, "%"
                DISPLAY "Recommendation: Switch traffic from ", link,
                    " to ", backup_link[link]
                REDIRECT_TRAFFIC(link, backup_link[link])
                LOG_EVENT(link, usage, "Switched to backup")
```

```

        ELSE
            DISPLAY link, " is operating normally at ", usage, "%"
        END IF

    END FOR

    // Step 6: Pause before next polling cycle
    WAIT FOR POLL_INTERVAL
END WHILE
END

```

2.3 Sample Monitoring Snapshot

Link	Utilization (%)	Threshold Check (>80%)	Action Taken
L1 (HQ-Branch A)	45	No	Continue on primary link
L2 (HQ-Branch B)	88	Yes	Alert raised; switched to Backup-L2
L3 (HQ-Branch C)	80	No (not strictly > 80)	Continue on primary link
L4 (HQ-Branch D)	92	Yes	Alert raised; switched to Backup-L4

2.4 Explanation — How the Algorithm Supports Bandwidth Management

The algorithm is designed as an infinite monitoring loop (WHILE monitoring_active) because network utilization is a dynamic quantity that must be observed continuously rather than as a one-time measurement. On every iteration, it walks through each link using a FOR EACH loop and records the current utilization reading into a list, which builds a historical dataset that can later be used for trend analysis, capacity planning, and reporting. The core decision-making happens in the IF condition: whenever utilization exceeds the 80% threshold, the link is flagged as overloaded, an alert is generated for the administrator, and traffic is automatically redirected to a pre-mapped backup link using the REDIRECT_TRAFFIC procedure. This prevents the overloaded link from degrading further and avoids packet drops, increased latency, and jitter that would otherwise occur if the link stayed saturated. Because the loop pauses for a fixed POLL_INTERVAL before repeating, the algorithm balances responsiveness (detecting overload quickly) against overhead (not polling so often that the monitoring itself consumes network resources). Overall, this proactive, threshold-based, self-healing design prevents congestion before it affects end users and keeps the organization's inter-branch communication reliable.

Question 3

A company has six branch offices connected through multiple network links, each with different and changing bandwidth, delay, and utilization values. Write a pseudocode algorithm that periodically collects these metrics, computes a link-quality score, selects the best path to each branch, detects overloaded/failed links, automatically switches to an alternate path, and alerts the administrator.

3.1 Problem Understanding

- Given (Inputs): Six branches (B1–B6), each reachable via one or more links; each link has bandwidth, delay, and utilization values that change over time.
- Required (Outputs): The best path to every branch (based on an overall quality score, not just distance), automatic failover on overload/failure, and an administrator alert.
- Constraints: Metrics must be collected periodically (a loop), scores must combine three normalized metrics, and overloaded/failed links must be excluded automatically.
- Approach: A weighted scoring function combines normalized bandwidth, delay, and utilization into a single comparable "link quality score", and the path with the maximum score is selected as the best path.

3.2 Pseudocode

```
ALGORITHM SelectBestPathByLinkQuality
BEGIN
    // Step 1: Initialization
    DECLARE branches[] = {B1, B2, B3, B4, B5, B6}
    DECLARE bandwidth[link], delay[link], utilization[link] // per-link metrics
    DECLARE quality_score[link]
    DECLARE best_path[branch]
    CONSTANT W_BW = 0.5, W_DELAY = 0.3, W_UTIL = 0.2 // weights (sum = 1.0)
    CONSTANT OVERLOAD_LIMIT = 80 // percentage
    CONSTANT MAX_BANDWIDTH, MAX_DELAY // normalization limits
    CONSTANT POLL_INTERVAL = 5 minutes
    system_running = TRUE

    // Step 2: Continuous monitoring loop
    WHILE system_running = TRUE DO
        FOR EACH branch IN branches DO

            FOR EACH link CONNECTING HeadOffice TO branch DO

                // Step 3: Collect live metrics for this link
                bandwidth[link] = MEASURE_BANDWIDTH(link)
                delay[link] = MEASURE_DELAY(link)
                utilization[link] = MEASURE_UTILIZATION(link)
                link_status[link] = CHECK_LINK_STATUS(link) // UP or DOWN

                // Step 4: Detect overloaded or failed links FIRST
                IF link_status[link] = "DOWN" OR utilization[link] >
OVERLOAD_LIMIT THEN
                    DISPLAY "ALERT: Link ", link, " to ", branch, " is
```

```

OVERLOADED/FAILED"
    quality_score[link] = -1          // disqualify this link
    NOTIFY_ADMIN(branch, link, "overloaded/failed - excluded")
ELSE
    // Step 5: Normalize metrics to a 0-1 scale
    norm_bw    = bandwidth[link] / MAX_BANDWIDTH
    norm_delay = 1 - (delay[link] / MAX_DELAY)    // lower
delay = better
    norm_util  = 1 - (utilization[link] / 100)    // lower
utilization = better

    // Step 6: Compute weighted link-quality score
    quality_score[link] = (W_BW * norm_bw) + (W_DELAY *
norm_delay)
                                + (W_UTIL * norm_util)
END IF

END FOR    // end links for this branch

// Step 7: Select the link with the highest quality score
best_path[branch] = LINK WITH MAXIMUM quality_score AMONG
                    links CONNECTING HeadOffice TO branch

IF best_path[branch] WAS CHANGED SINCE LAST CYCLE THEN
    DISPLAY "Path to ", branch, " switched to ", best_path[branch]
    NOTIFY_ADMIN(branch, best_path[branch], "path changed")
END IF

DISPLAY "Best path to ", branch, " = ", best_path[branch],
        " | Score = ", quality_score[best_path[branch]]

END FOR    // end all branches

// Step 8: Wait before the next monitoring cycle
WAIT FOR POLL_INTERVAL
END WHILE
END

```

3.3 Sample Scoring Snapshot for One Branch (B2)

Link	Bandwidth	Delay (ms)	Utilization (%)	Status	Quality Score	Selected
Link A	80 Mbps	20	55	UP	0.71	Yes (Best)
Link B	60 Mbps	35	90	UP	Excluded (>80% util)	No
Link C (Backup)	40 Mbps	45	30	UP	0.58	No

3.4 Explanation — Reliability, Efficiency, and Fault Tolerance

Unlike a simple shortest-path or fewest-hops approach, this algorithm continuously re-evaluates every candidate link using three live metrics — bandwidth, delay, and utilization — combined into a single weighted quality score. This ensures the path chosen is the one that is actually performing best at that moment, not merely the one that is topologically shortest. Normalizing each metric to a common 0–1 range before applying weights allows quantities with very different units and scales (megabits per second, milliseconds, percentage) to be compared fairly, and the weights (W_{BW} , W_{DELAY} , W_{UTIL}) let the administrator tune how much importance is given to raw capacity versus responsiveness versus current load, depending on the organization's priorities.

Fault tolerance is built in through the overload/failure check performed before scoring: any link that is down or already utilizing more than 80% of its capacity is immediately disqualified (score = -1) and excluded from selection, and an alert is sent to the administrator so the issue can be investigated. Because the outer WHILE loop repeats this entire evaluation at fixed intervals, the system automatically detects when a previously best link degrades or fails and reroutes traffic to the next-best alternative without manual intervention, which is the essence of self-healing, adaptive routing. This continuous re-scoring also improves routing efficiency, since traffic is always steered along the path with the best real-time balance of speed, responsiveness, and available capacity, rather than a path that was optimal only when the network was first configured. Together, these mechanisms make the six-branch network more reliable, more efficient, and resilient to the changing conditions that occur throughout a business day.

Rubrics for Evaluation

Criteria	Weight age	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient